

```

56. void doubleVal(int val)
    {
        *val *= 2;
    }
57. int *pint = nullptr;
    new pint;
58. int *pint = nullptr;
    pint = new int;
    if (pint == nullptr)
        *pint = 100;
    else
        cout << "Memory allocation error\n";
59. int *pint = nullptr;
    pint = new int[100]; // Allocate memory
    .
    .
    Code that processes the array.
    .
    .
    delete pint; // Free memory
60. int *getNum()
    {
        int wholeNum;
        cout << "Enter a number: ";
        cin >> wholeNum;
        return &wholeNum;
    }
61. const int arr[] = { 1, 2, 3 };
    int *ptr = arr;
62. void doSomething(int * const ptr)
    {
        int localArray[] = { 1, 2, 3 };
        ptr = localArray;
    }

```

Programming Challenges

1. Array Allocator

Write a function that dynamically allocates an array of integers. The function should accept an integer argument indicating the number of elements to allocate. The function should return a pointer to the array.

2. Test Scores #1

Write a program that dynamically allocates an array large enough to hold a user-defined number of test scores. Once all the scores are entered, the array should be passed to a function that sorts them in ascending order. Another function should be

called that calculates the average score. The program should display the sorted list of scores and averages with appropriate headings. Use pointer notation rather than array notation whenever possible.

Input Validation: Do not accept negative numbers for test scores.

3. Drop Lowest Score

Modify Problem 2 above so the lowest test score is dropped. This score should not be included in the calculation of the average.

4. Test Scores #2

Modify the program of Programming Challenge 2 to allow the user to enter name-score pairs. For each student taking a test, the user types the student's name followed by the student's integer test score. Modify the sorting function so it takes an array holding the student names and an array holding the student test scores. When the sorted list of scores is displayed, each student's name should be displayed along with his or her score. In stepping through the arrays, use pointers rather than array subscripts.

5. Pointer Rewrite

The following function uses reference variables as parameters. Rewrite the function so it uses pointers instead of reference variables, and then demonstrate the function in a complete program.

```
int doSomething(int &x, int &y)
{
    int temp = x;
    x = y * 10;
    y = temp * 10;
    return x + y;
}
```

6. Case Study Modification #1

Modify Program 9-19 (the United Cause case study program) so it can be used with any set of donations. The program should dynamically allocate the `donations` array and ask the user to input its values.

7. Case Study Modification #2

Modify Program 9-19 (the United Cause case study program) so the `arrptr` array is sorted in descending order instead of ascending order.

8. Mode Function

In statistics, the *mode* of a set of values is the value that occurs most often or with the greatest frequency. Write a function that accepts as arguments the following:

- A) An array of integers
- B) An integer that indicates the number of elements in the array

The function should determine the mode of the array. That is, it should determine which value in the array occurs most often. The mode is the value the function should return. If the array has no mode (none of the values occur more than once), the function should return `-1`. (Assume the array will always contain nonnegative values.)

Demonstrate your pointer prowess by using pointer notation instead of array notation in this function.



VideoNote
Solving the
Pointer Rewrite
Problem

9. Median Function

In statistics, when a set of values is sorted in ascending or descending order, its *median* is the middle value. If the set contains an even number of values, the median is the mean, or average, of the two middle values. Write a function that accepts as arguments the following:

- A) An array of integers
- B) An integer that indicates the number of elements in the array

The function should determine the median of the array. This value should be returned as a `double`. (Assume the values in the array are already sorted.)

Demonstrate your pointer prowess by using pointer notation instead of array notation in this function.

10. Reverse Array

Write a function that accepts an `int` array and the array's size as arguments. The function should create a copy of the array, except that the element values should be reversed in the copy. The function should return a pointer to the new array. Demonstrate the function in a complete program.

11. Array Expander

Write a function that accepts an `int` array and the array's size as arguments. The function should create a new array that is twice the size of the argument array. The function should copy the contents of the argument array to the new array and initialize the unused elements of the second array with 0. The function should return a pointer to the new array.

12. Element Shifter

Write a function that accepts an `int` array and the array's size as arguments. The function should create a new array that is one element larger than the argument array. The first element of the new array should be set to 0. Element 0 of the argument array should be copied to element 1 of the new array, element 1 of the argument array should be copied to element 2 of the new array, and so forth. The function should return a pointer to the new array.

13. Movie Statistics

Write a program that can be used to gather statistical data about the number of movies college students see in a month. The program should perform the following steps:

- A) Ask the user how many students were surveyed. An array of integers with this many elements should then be dynamically allocated.
- B) Allow the user to enter the number of movies each student saw into the array.
- C) Calculate and display the average, median, and mode of the values entered. (Use the functions you wrote in Problems 8 and 9 to calculate the median and mode.)

Input Validation: Do not accept negative numbers for input.